

전자전기컴퓨터설계실험II

실험 후 보고서

LAB5

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 10월 24일
대상	인터럽트와 PWM
검증 환경	VS Code / STM32 / Vivado / FPGA

1. 실험 목적과 관련 이론

실험 목적

STM32의 버튼 입력을 인터럽트로 처리하고 타이머로 LED 점멸 및 PWM을 구현하였다. 과제 3에서는 1→2→4→8→1초 간격 변경, 과제 4에서는 25→50→75→25% 듀티비 변경을 확인하였다. FPGA에서는 피아노, 신호등, ADC를 구현하고 시뮬레이션과 보드 동작을 비교하였다.

폴링과 인터럽트

PC13 버튼의 상승 에지에서 버튼 ISR을 실행하고, TIM6 갱신 인터럽트에서 LED의 반전 시점을 결정하였다. polling_routine()은 비워 두어 버튼 검사와 시간 판정을 메인 루프에서 반복하지 않도록 하였다. HAL 콜백은 인터럽트 처리와 실습 함수를 연결한다.[1]

인터럽트 우선순위와 공유 변수

TIM6의 우선순위는 4, 버튼은 7로 설정되어 있다. 버튼 ISR에서 간격과 카운터를 바꾸는 동안 TIM6가 선점할 수 있으므로 두 값을 함께 갱신하는 부분을 짧은 임계 구역으로 보호하였다. volatile은 값의 접근을 보장하기 위한 선언이며 여러 연산의 원자성을 보장하지 않는다

타이머와 PWM

TIM6는 100 ms마다 기준 이벤트를 발생시키고, TIM3 CH1은 PB4로 1 kHz PWM을 출력한다. PWM 듀티비는 D=tH/T×100%이다. 주기는 유지한 채 비교값 CCR1을 바꾸면 HIGH 시간만 변한다.[1]

실험 장치

구분	구성
STM32	NUCLEO-G474RE
FPGA	HBE-COMBO2-DLD xc7s75fgga484-1
개발	VS Code, Vivado
계측	Tektronix TBS2000B

2. 과제 3 인터럽트 LED

설계 목표와 회로 구성

PC13 USER 버튼과 PA5 내장 LED를 사용하였다. 버튼을 한 번 누르면 상태 변경 간격을 한 단계씩 바꾸고, 입력이 없으면 현재 간격을 유지하도록 설계하였다 LED의 켜짐과 꺼짐이 각각 한 구간이므로 전체 점멸 주기는 설정 간격의 두배이다.[2]

```
fTIM6 = 100,000,000 Hz
PSC = 9999, ARR = 999
T = (PSC+1)(ARR+1)/fTIM6
  = 0.1 s = 100 ms
```

초기화와 타이머 ISR

```
__HAL_TIM_SET_COUNTER(&htim6,0U);
__HAL_TIM_CLEAR_FLAG(
    &htim6, TIM_FLAG_UPDATE);
HAL_TIM_Base_Start_IT(&htim6);
```

초기 갱신 플래그를 지운 후 인터럽트를 시작하였다. 초기 간격은 1000 ms이고 카운터는 0이다. 아래는 tim6_isr_p4()의 핵심 처리이다. 오류 처리와 계측 카운터는 발췌에서 생략하였다.

```
++tim6_count;
if (tim6_count >=
    blink_interval_ms / 100U)
{
    tim6_count = 0U;
    HAL_GPIO_TogglePin(
        LED_ece2_GPIO_Port,
        LED_ece2_Pin);
}
```

버튼 입력 처리

첫 누름은 바로 인정하고 마지막 유효 입력 후 200 ms 이내의 반복 에지는 무시하였다. 이는 접점 바운스에 의한 중복 변경을 줄인다. 유효 입력마다 간격을 두 배로 바꾸고 8초 다음에는 1초로 복귀시켰다.

간격 변경 핵심 코드

```
uint32_t saved_primask =
    __get_PRIMASK();
__disable_irq();
blink_interval_ms =
    (blink_interval_ms >= 8000U)
    ? 1000U
    : blink_interval_ms * 2U;
tim6_count = 0U;
__set_PRIMASK(saved_primask);
```

예상 결과와 실제 결과

설정 간격	TIM6 횟수	전체 주기
1 s	10	2 s
2 s	20	4 s
4 s	40	8 s
8 s	80	16 s

보드에서 버튼 입력에 따라 1→2→4→8→1초 순서로 LED 상태 변경 간격이 순환하였고, 버튼을 누르지 않으면 선택된 간격을 유지하였다. 이로써 과제 3의 요구 동작을 확인하였다. 위 표의 시간은 설정값에 따른 계산값이며 정밀 계측값은 아니다.

디버깅 관찰과 해석

버튼 ISR 중단점에 도달한 것은 EXTI 처리 경로로 진입했다는 근거이다. 단, 간격 변경 문장보다 앞에서 멈추면 Watch에 기존 값 1000이 표시될 수 있다. F10으로 대입문을 실행한 뒤 새 값을 확인하고 F5로 연속 실행을 재개한다.

중단점 정지 중의 LED 정지는 정상 실행의 고장과 구분하였다. 시간 간격을 판단할 때는 중단점을 해제한 연속 동작을 기준으로 하였다. 간격 변경 시 tim6_count만 0으로 바꾸므로, 변경 직후 첫 반전은 TIM6의 남은 100 ms 구간에 영향을 받을 수 있다.

실험 결과

동영상으로 실험 결과를 판별할 수 있어서 동영상 첨부로 실험 결과를 대신하였다.
https://github.com/thechef030/STM32_1/tree/stm32-1/evidence/board
(동영상 제목: 과제3_인터럽트_LED_video)

3. 과제 4 PWM 비율 변경

설계 목표와 회로 구성

PC13 버튼으로 PWM을 25→50→75→25% 순서로 변경하였다. PB4의 TIM3 CH1 출력을 오실로스코프 CH1에 연결하고 보드 GND를 공통 기준으로 사용하였다. PB4는 Arduino 헤더의 D5이며 PA5 내장 LED 출력과 다르다.[2]

```
fTIM3 = 100,000,000 Hz
PSC = 99, ARR = 999
fPWM = fTIM3
    / ((PSC+1)*(ARR+1))
     = 1000 Hz
D = CCR1 / (ARR+1)
T = 1 ms
```

초기화 핵심 코드

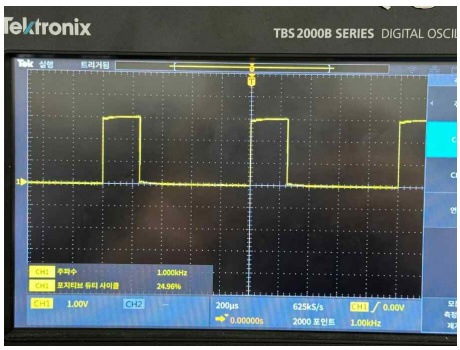
```
pwm_percent = 25U;
pwm_compare = 250U;
__HAL_TIM_SET_COMPARE(
    &htim3, TIM_CHANNEL_1,
    pwm_compare);
HAL_TIM_PWM_Start(
    &htim3, TIM_CHANNEL_1);
```

버튼 ISR 핵심 코드

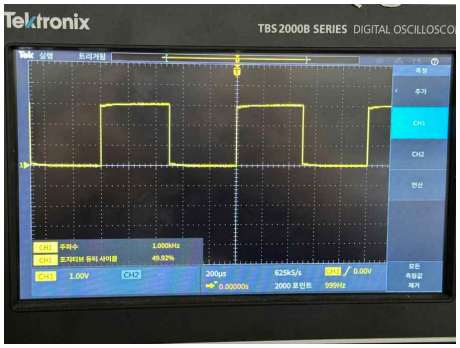
```
pwm_percent =
    (pwm_percent >= 75U)
    ? 25U : pwm_percent + 25U;
pwm_compare = pwm_percent * 10U;
__HAL_TIM_SET_COMPARE(
    &htim3, TIM_CHANNEL_1,
    pwm_compare);
```

공통 입력 억제 처리 후 위 코드를 실행하였다. ARR+1=1000이므로 CCR1을 250, 500, 750으로 바꾸면 각각 25%, 50%, 75%가 된다. TIM3 하드웨어가 파형을 계속 생성하며 메인 루프에서 GPIO를 직접 반전시키지 않는다.[1][2]

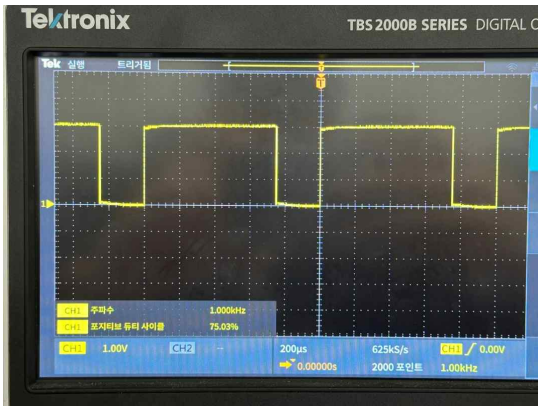
오실로스코프 측정 결과



- 목표 25%에서 측정한 PWM. 듀티비 24.96%, 주파수 1.000 kHz.



- 목표 50%에서 측정한 PWM. 듀티비 49.92%, 주파수 1.000 kHz.



- 목표 75%에서 측정한 PWM. 듀티비 75.03%, 주파수 1.000 kHz.

세 화면 모두 시간축은 200 µs/div, CH1은 1.00 V/div이다. 주기가 거의 일정한 상태에서 HIGH 구간이 약 1/4, 1/2, 3/4로 증가하였다. 사진의 측정값을 다음 표에 정리하였다.

목표값과 측정값 비교

목표	측정	차이 %p
25%	24.96%	−0.04
50%	49.92%	−0.08
75%	75.03%	+0.03

목표	주파수	tH 환산
25%	1.000 kHz	249.6 µs
50%	1.000 kHz	499.2 µs
75%	1.000 kHz	750.3 µs

HIGH 시간은 표시 주파수로 T=1 ms를 구한 뒤 측정 듀티비를 곱한 환산값이다. 별도 커서 측정값은 아니다. 목표 대비 상대 오차는 순서대로 0.16%, 0.16%, 0.04%이며, 절대 편차의 최댓값은 0.08%p이다. 50% 사진의 하단 주파수

표시에는 999 Hz도 보이지만 CH1 자동 측정 항목은 1.000 kHz이다. 표는 세 사진의 동일한 CH1 측정 항목을 기준으로 정리하였다. 표시 갱신과 측정 분해능을 고려하면 약 1 kHz의 주파수가 유지된 결과이다.

C 디버거 확인

button_isr_p7()의 PWM 변경 부분에 중단점을 두고 버튼 입력에 따른 함수 진입을 확인하였다. F10 단위 실행으로 변수와 비교값의 변화를 확인한 후 중단점을 해제하여 실제 출력 파형을 측정하였다. 기능 및 디버깅 절차는 정상 동작하였다.

상태	설정 변수	비교값
시작	25	250
1회 누름	50	500
2회 누름	75	750
3회 누름	25	250

표는 코드의 설정 순서이다. 오실로스코프 결과는 이 세 상태의 듀티비와 일치하였다.

실험 결과

동영상 첨부와 위에 첨부한 오실로스코프 사진을 실험 결과로 업로드하였다.
https://github.com/thechef030/STM32_1/tree/stm32-1/evidence/board
(동영상 제목:
과제4_버튼으로_PWM비율변경_video)

4. 피아노

피아노 설계 목표와 회로 구조

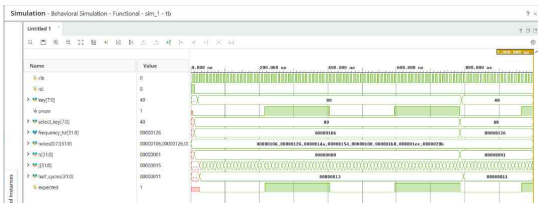
8개 키 입력에 따라 음계를 선택하고 피에조에 사각파를 출력하였다.

note_select는 키를 주파수로 변환하고 square_tone은 카운터로 반주기를 생성한다. piano는 두 모듈을 연결한다. 키가 없으면 출력을 정지시키며 여러 키 입력의 우선순위는 코드에 따른다.

```
half_cycles =
    CLK_HZ / (2 * frequency_hz);
```

기본 보드 클록은 50 MHz이다.
테스트벤치는 짧은 시간에 음의 변화를 검사하도록 분주 계산용 CLK_HZ를 줄인다. 따라서 시뮬레이션의 ns 단위 파형 주기를 실제 음의 주기로 해석하지 않았다.

Vivado 실행 결과



신호	첫 음	다음 음
key	0x80	0x40
frequency_hz	0x106 262 Hz	0x126 294 Hz
half_cycles	0x13 19	0x11 17

키가 0x80에서 0x40으로 바뀌면서 선택 주파수가 262 Hz에서 294 Hz로 증가하고 반주기 카운트는 19에서 17로 감소하였다. 표시 구간에서 piezo와 expected의 전환이 대응한다.

피아노 비트스트림 생성 결과



비트스트림 생성이 완료되었으며 표시된 설정 시간과 유지 시간 여유는 양수이다. 실제 보드에서도 피아노 동작에 성공하였다

실험 결과

동영상으로 실험 결과를 판별할 수 있어서 동영상 첨부로 실험 결과를 대신하였다.

https://github.com/thechef030/STM32_1/tree/stm32-1/evidence/board

(동영상 제목: piano_video)

5. 신호등 제어

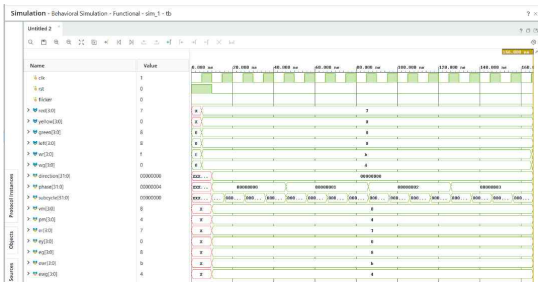
신호등 설계 목표와 회로 구조

second_tick으로 기준 시간과 점멸 신호를 만들고, traffic_sequence로 방향과 단계를 진행시켰다. traffic_lights가 차량·보행자 LED를 선택한다. 방향 비트 [3:0]은 남·북·서·동에 대응한다.

```
if (rst_p || flicker) begin
    direction <= 0;
    phase <= 0;
end else if (tick) begin
    if (phase == 12) begin
        phase <= 0;
        direction <= direction+1;
    end else
        phase <= phase+1;
end
```

한 방향은 13단계이며 네 방향의 전체 순환은 52초이다. 차량 초록과 좌회전은 함께 켜진다. flicker가 HIGH이면 일반 진행을 초기 방향에 두고 차량 노랑과 보행자 빨강을 점멸시킨다.[1]

Vivado 실행 결과



출력	표시값	해석
red	0x7	남쪽 외 빨강
yellow	0x0	노랑 꺼짐
green / left	0x8	남쪽 켜짐
wr / wg	0xB / 0x4	선택 보행자 초록

리셋 이후 direction=0에서 남쪽 차량 초록과 좌회전이 켜지고, phase가 0, 1, 2, 3으로 진행한다. 해당 구간에서 red=7, green=8, left=8, wr=B, wg=4가 기대값 er·eg·ewr·ewg와 대응하였다.

오른쪽 끝은 phase=4로 넘어가는 경계이며 Value에는 wg=0, wr=F가 표시된다. 화면의 이전 구간과 끝점 값을 구분해야 한다.

단계별 예상 동작

단계	차량	보행자
0~3	초록·좌회전	초록 유지
4~10	초록·좌회전	초록 점멸
11~12	노랑	빨강

이 파형은 초기 방향의 일부 구간을 보여준다. 전체 52초 순환과 점멸 모드의 보드 동작은 실험에서 정상적으로 확인하였다. 시뮬레이션은 CLK_HZ=4로 가속되므로 화면 시간축을 실제 초 단위로 환산하지 않았다.

신호등 비트스트림 생성 결과



신호등 비트스트림 생성이 완료되었고 보드 동작에도 성공하였다. 기능 상태를 결정하는 논리와 실제 LED 핀 연결을 구분하여 XDC를 적용하였다. 일반 LED 8개 핀 표를 신호등 전용 출력에 그대로

적용하지 않았다.

실험 결과

동영상으로 실험 결과를 판별할 수 있어서 동영상 첨부로 실험 결과를 대신하였다.
https://github.com/thechef030/STM32_1/tree/stm32-1/evidence/board
(동영상 제목: traffic_control_video)

6. ADC 제어

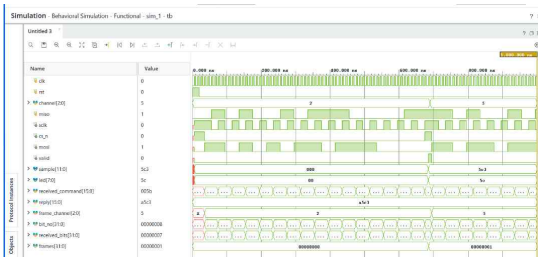
ADC 설계 목표와 회로 구조

FPGA에서 외부 ADC를 직렬 인터페이스로 제어하고 수신 데이터를 LED에 표시하였다. adc_command는 선택 채널에 맞는 명령을 만들고 serial_frame16은 16비트 송수신을 담당한다. adc는 변환값과 완료 신호를 갱신한다. STM32 내부 ADC를 사용하는 실습은 아니다.

```
if (done)
    sample <= rx_word[11:0];
assign led_out = sample[11:4];
```

이번 코드의 sample은 12비트, LED는 상위 8비트이다. 이는 수신 워드의 디지털 해석 방식이다.

Vivado 실행 결과



항목	확인값
응답 reply	0xA5C3
sample[11:0]	0x5C3 = 1475
led[7:0]	0x5C = 92
채널	2 → 5
frames	1

약 0.68 μs 부근에서 첫 프레임이 끝나고 valid 펄스가 발생하며 sample이 000에서 5C3으로, led가 00에서 5C로 바뀐다.
응답 0xA5C3의 하위 12비트와 그 상위 8비트를 취하는 코드와 일치한다.
완료 후 채널은 2에서 5로 바뀌며 다음 프레임이 시작된다.

실제 보드 결과와 검증 범위

외부 ADC를 이용한 보드 동작은 정상적으로 확인하였다. 시뮬레이션의 응답값은 테스트벤치가 제공한 디지털 데이터이며 실제 센서 전압 측정값은 아니다.

기능 검증에서는 CS 활성 구간, 직렬 클록, 송수신 비트 및 완료 신호와 결과 갱신 관계를 구분하였다. 입력 채널을 바꿀 때 반환되는 데이터가 어느 변환에 해당하는지는 실제 ADC의 파이프라인 조건을 따라야 한다.

ADC 비트스트림 생성 결과



실험 결과

동영상으로 실험 결과를 판별할 수 있어서 동영상 첨부로 실험 결과를 대신하였다.
https://github.com/thechef030/STM32_1/tree/stm32-1/evidence/board
(동영상 제목: ADC_video)

7. 소스 구성과 GITHUB저장소

파일	역할
ece2_474_routine.c	초기화·버튼·TIM6 ISR
main.c	GPIO·타이머 설정
CMake 설정	과제 3·4 빌드 선택
FPGA_additional.md	피아노·신호등·ADC 설계 및 테스트벤치

브랜치와 버전 확인

-github주소:
https://github.com/thechef030/STM32_1

- 브랜치: stm32-1
- 최신 커밋 전체 해시: 7daa24ad894ba4ba2180ccf7005a36134aac31b8
- 등록 태그: v1.1.0
- 태그가 가리키는 전체 커밋 해시: 22cdb6b7dae93de8506f77084e07b34bbd36a5c8

v1.1.0은 최신 HEAD와 다른 커밋이다.

8. 참고문헌

전전설II LAB5 교안

[1] STMicroelectronics, STM32CubeG4 MCU Firmware Package 및 STM32G4 HAL TIM 드라이버. 타이머 인터럽트 시작, PWM 시작 및 타이머 제어 코드 참조.

[2] STMicroelectronics, STM32G4 Nucleo-64 boards (MB1367), UM2505. NUCLEO-G474RE의 버튼, LED와 헤더 핀 연결 참조.